

Interfaces em Java

Programando para abstração, não para implementação

Um problema para começar

Uma loja quer aceitar várias formas de pagamento:

- **Boleto** – gera um código de barras e um vencimento;
- **Cartão de Crédito** – processa a cobrança em uma operadora, podendo parcelar;
- **Pix** – gera uma chave e confirma a transferência na hora.

Apesar de serem bem diferentes entre si, todas essas formas de pagamento precisam fazer a mesma coisa: **processar um pagamento**.

Será que devemos usar herança aqui?

Já aprendemos que, quando classes compartilham comportamento, podemos usar herança.

Mas pense bem...

Faz sentido dizer que `Boleto` `extends` `CartaoCredito`? Ou que `Pix` `extends` `Boleto`?

- `Boleto`, `Cartão de Crédito` e `Pix` **não são tipos especializados um do outro**;
- eles não compartilham atributos nem comportamento em comum, exceto uma coisa: todos processam um pagamento.

Herança representa uma relação "**é um**". Aqui, essa relação não existe entre essas classes.

E se criássemos uma superclasse Pagamento?

Ainda resta uma ideia tentadora: em vez de uma classe herdar da outra, criar uma **superclasse comum** para as três:

```
public abstract class Pagamento {  
    public abstract void pagar(double valor);  
}  
  
public class Boleto extends Pagamento { /* ... */ }  
public class CartaoCredito extends Pagamento { /* ... */ }  
public class Pix extends Pagamento { /* ... */ }
```

Atenção

Diferente da tentativa anterior, esse código **compila e funciona perfeitamente**. É por isso que essa armadilha é mais traiçoeira que a primeira.

Por que essa ideia parece fazer sentido

- em português, soa natural dizer "Boleto é **um** Pagamento", "Cartão é **um** Pagamento";
- o código compila, os métodos funcionam, o polimorfismo funciona normalmente – então qual seria o problema?

O problema não está na sintaxe

O problema está no **significado** da relação que essa herança estaria criando.

O que é, de fato, um Pagamento?

- um **Pagamento** é um **evento**: acontece em uma data, tem um valor, tem um status (pago, pendente, recusado);
- Boleto, CartaoCredito e Pix **não são** esse evento – são os **meios** usados para realizá-lo.

A frase certa

Não é "Boleto é **um** Pagamento". É "Boleto **processa** um pagamento" – ele tem a **capacidade** de pagar, não é o próprio pagamento.

Mesmo funcionando, herança seria a ferramenta errada

- a superclasse Pagamento não teria nenhum atributo nem método com corpo compartilhado – ela existiria só para forçar um tipo em comum;
- em Java, uma classe só pode ter extends de **uma** superclasse. Se um dia Boleto também precisasse ser um Documento ou um ComprovanteFiscal, essa herança já estaria "gasta" com Pagamento;
- quando uma "superclasse" só serve para garantir um método em comum, sem nenhum código compartilhado, ela não é uma hierarquia de tipos – é um **contrato** disfarçado de herança.

Provando na prática: a herança única trava você

Imagine que, além de pagar, um Boleto também precisasse saber gerarPDF(), por ser um tipo de documento:

```
public class Documento {  
    public void gerarPDF() { /* ... */ }  
}  
  
// Boleto ja "e um" Pagamento... e agora?  
public class Boleto extends Pagamento, Documento {  
    // ERRO DE COMPILACAO!  
    // Java nao permite "extends" de duas classes.  
}
```

O que aconteceu

Boleto já "gastou" sua única herança em Pagamento – não sobrou espaço para também herdar de Documento.

Provando na prática: a superclasse vira um emaranhado de ifs

E se quiséssemos aproveitar Pagamento para algo além da assinatura, como imprimir um comprovante?

```
public abstract class Pagamento {  
    public void imprimirComprovante() {  
        if (this instanceof Boleto) {  
            // dados do boleto...  
        } else if (this instanceof CartaoCredito) {  
            // dados do cartao...  
        } else if (this instanceof Pix) {  
            // dados do pix...  
        }  
    }  
    public abstract void pagar(double valor);  
}
```

Uma superclasse que precisa perguntar "quem você é"

Usar `instanceof` dentro de `Pagamento` é a classe admitindo que **não sabe lidar** com suas subclasses de forma genérica – o oposto do que herança deveria oferecer.

O problema cresce

A cada nova forma de pagamento criada (`Cripto`, `Cheque...`), alguém precisa **voltar** nessa classe e adicionar mais um `else if`.

O que elas realmente compartilham

- Boleto, Cartão de Crédito e Pix **não são tipos especializados um do outro**, nem são tipos especializados de um "Pagamento" genérico;
- eles não compartilham atributos nem estado em comum, como vimos;
- a única coisa que todos têm em comum é um **comportamento**: todos sabem processar um pagamento.

Herança representa "é um"; aqui, é só "sabe fazer"

Não existe uma hierarquia de tipos legítima entre essas classes – então herança não é a ferramenta certa para este problema.

O que realmente precisamos

Não precisamos que essas classes compartilhem **código**.

Precisamos apenas garantir que todas elas **prometam** fazer uma coisa: ter um método para processar o pagamento.

Ideia central

E se, em vez de uma relação de herança, existisse uma forma de apenas **obrigar** classes completamente diferentes a implementar um mesmo método, com o mesmo nome e assinatura?

Essa ferramenta existe em Java e se chama **interface**.

O que é uma interface?

Definição

Uma **interface** é um tipo que define **um conjunto de métodos que uma classe promete implementar**, sem dizer **como** esses métodos devem ser implementados.

- Uma interface declara **o que** deve ser feito;
- cada classe que implementa a interface decide **como** fazer;
- por isso, dizemos que uma interface é um **contrato**.

O que é um contrato?

Analogia

Pense em um contrato de prestação de serviço: ele lista **quais obrigações** devem ser cumpridas, mas não diz **como** cada pessoa vai cumpri-las.

Da mesma forma, uma interface Java diz:

- "toda classe que assinar este contrato precisa ter um método `pagar(double valor)`";
- mas não diz se o pagamento será feito por boleto, cartão ou Pix – isso fica por conta de quem assina o contrato.

Se a classe assina o contrato (`implements`) e não cumpre uma das obrigações, o código **não compila**.

Sintaxe: declarando uma interface

Uma interface é declarada com a palavra-chave `interface`, no lugar de `class`:

```
public interface Pagavel {  
    void pagar(double valor);  
}
```

- o método `pagar` não tem corpo (não tem chaves com código dentro), apenas assinatura;
- toda classe que implementar `Pagavel` será **obrigada** a fornecer uma implementação para `pagar(double valor)`.

Implementando uma interface: Boleto

Uma classe assina o contrato com a palavra-chave `implements`:

```
public class Boleto implements Pagavel {  
  
    private String codigoBarras;  
  
    public Boleto(String codigoBarras) {  
        this.codigoBarras = codigoBarras;  
    }  
  
    @Override  
    public void pagar(double valor) {  
        System.out.println("Pagando R$ " + valor  
            + " via boleto " + codigoBarras);  
    }  
}
```


Implementando a mesma interface: CartaoCredito e Pix

```
public class CartaoCredito implements Pagavel {
    private String numero;

    public CartaoCredito(String numero) {
        this.numero = numero;
    }

    @Override
    public void pagar(double valor) {
        System.out.println("Cobrando R$ " + valor
            + " no cartão " + numero);
    }
}

public class Pix implements Pagavel {
    private String chave;

    public Pix(String chave) {
        this.chave = chave;
    }

    @Override
    public void pagar(double valor) {
        System.out.println("Transferindo R$ " + valor
```

Regras básicas de uma interface

- os métodos de uma interface são, por padrão, **públicos e abstratos** (sem corpo);
- uma interface **não possui atributos de instância** – ou seja, não guarda estado como uma classe normal;
- uma classe usa `implements` (em vez de `extends`) para assinar o contrato;
- se a classe não implementar **todos** os métodos da interface, o código não compila.

Diferença importante em relação a herança

Herança (`extends`) representa "é um". Interface (`implements`) representa "sabe fazer" ou "cumpre este contrato".

Uma classe pode implementar várias interfaces

Diferente da herança de classes (onde só existe **uma** superclasse direta), uma classe pode assinar **vários contratos ao mesmo tempo**:

```
public interface Pagavel {  
    void pagar(double valor);  
}  
  
public interface Cancelavel {  
    void cancelar();  
}  
  
public class CartaoCredito implements Pagavel, Cancelavel {  
    @Override  
    public void pagar(double valor) { /* ... */ }  
  
    @Override  
    public void cancelar() { /* ... */ }  
}
```

CartaoCredito agora promete cumprir **dois** contratos diferentes.

Polimorfismo com interfaces

Assim como acontece com herança, uma variável do tipo da interface pode referenciar qualquer classe que a implemente:

```
Pagavel forma1 = new Boleto("00190.00009");  
Pagavel forma2 = new CartaoCredito("1234-5678");  
Pagavel forma3 = new Pix("loja@email.com");  
  
forma1.pagar(100.0); // "Pagando R$ 100.0 via boleto..."  
forma2.pagar(250.0); // "Cobrando R$ 250.0 no cartão..."  
forma3.pagar(50.0);  // "Transferindo R$ 50.0 via Pix..."
```

Observação

O tipo da variável é `Pagavel` nos três casos, mas cada objeto executa a sua **própria** implementação de `pagar()` – o mesmo polimorfismo que já vimos com herança, agora através de uma interface.

Uma lista de formas de pagamento

Como todas essas classes cumprem o mesmo contrato, podemos guardá-las juntas em uma única lista:

```
List<Pagavel> formas = new ArrayList<>();
formas.add(new Boleto("00190.00009"));
formas.add(new CartaoCredito("1234-5678"));
formas.add(new Pix("loja@email.com"));

for (Pagavel forma : formas) {
    forma.pagar(100.0);
}
```

Ideia-chave

O for nem sabe (nem precisa saber) se cada forma é um boleto, um cartão ou um Pix. Ele só precisa saber que todo Pagavel tem um método pagar().

Programar para a abstração, não para a implementação

Princípio

Sempre que possível, escreva código que dependa de **interfaces** (o que um objeto sabe fazer), e não de **classes concretas** (como ele faz por dentro).

Já usamos esse princípio sem perceber: no exemplo anterior, o `for` dependia apenas de `Pagavel`, nunca de `Boleto`, `CartaoCredito` ou `Pix`.

Vamos ver esse mesmo princípio em uma interface que vocês já usam o tempo todo: `List`.

A interface List e suas implementações

List é uma interface do Java que representa "uma coleção ordenada de elementos". Ela define o contrato: `add()`, `get()`, `remove()`, `size()`, entre outros.

Várias classes assinam esse contrato, cada uma com uma forma diferente de guardar os dados por dentro:

- **ArrayList** – guarda os elementos em um array que cresce conforme necessário;
- **LinkedList** – guarda os elementos em uma lista encadeada (cada elemento aponta para o próximo);
- **Vector** – parecido com ArrayList, mas com segurança extra para uso concorrente.

Todas prometem exatamente os mesmos métodos, mas cada uma implementa por dentro do seu próprio jeito.

Duas formas de declarar a mesma lista

Compare estas duas declarações:

```
ArrayList<String> lista = new ArrayList<>();  
lista.add("Rex");
```

```
List<String> lista = new ArrayList<>();  
lista.add("Rex");
```

Diferença

Na primeira, a variável **só enxerga** o que existe em `ArrayList`. Na segunda, a variável enxerga apenas o que existe na interface `List` – mas o objeto real continua sendo um `ArrayList`.

Trocando a implementação sem quebrar o código

Se declaramos a variável como `List`, podemos trocar a implementação concreta **sem alterar mais nada**:

```
List<String> lista = new ArrayList<>();  
// List<String> lista = new LinkedList<>();  
// List<String> lista = new Vector<>();  
  
lista.add("Rex");  
lista.add("Mimi");  
lista.remove("Rex");  
System.out.println(lista.size());
```

Nenhuma dessas três últimas linhas muda, **não importa qual das três implementações você escolher** na primeira linha.

Por que isso funciona? O contrato da List

O contrato garante o comportamento esperado

A interface `List` garante que `add`, `get`, `remove` e `size` **sempre existirão**, não importa qual classe concreta você escolher.

- Se o seu código depende apenas de `List`, ele **não precisa saber** (nem se importar) qual implementação está sendo usada por baixo dos panos;
- você pode trocar `ArrayList` por `LinkedList` depois, por exemplo, para melhorar a performance de inserções no meio da lista, **sem reescrever o resto do sistema**.

Um método que aceita qualquer List

O mesmo raciocínio vale para métodos: recebendo List como parâmetro (e não ArrayList), o método funciona com **qualquer** implementação:

```
public static void imprimir(List<String> nomes) {  
    for (String nome : nomes) {  
        System.out.println(nome);  
    }  
}  
  
// Todas as chamadas abaixo funcionam:  
imprimir(new ArrayList<>());  
imprimir(new LinkedList<>());  
imprimir(new Vector<>());
```

Se o parâmetro fosse do tipo ArrayList, as duas últimas chamadas **não** compilariam.

Aplicando a mesma ideia ao pagamento

A mesma lógica se aplica ao nosso exemplo anterior:

```
public static void processarPagamento(Pagavel forma, double valor)
{
    forma.pagar(valor);
}

processarPagamento(new Boleto("00190.00009"), 100.0);
processarPagamento(new CartaoCredito("1234-5678"), 250.0);
processarPagamento(new Pix("loja@email.com"), 50.0);
```

Ideia-chave

Se a loja criar uma nova forma de pagamento no futuro (por exemplo, CriptoMoeda), esse método **não precisa mudar nada**, desde que a nova classe também implemente Pagavel.

Por que o nome de uma interface importa

Quando lemos `implements Pagavel`, `implements Comparable` ou `implements Runnable`, conseguimos entender o contrato **sem olhar a implementação**.

Ideia central

O nome de uma interface é a primeira – e às vezes única – documentação do contrato que ela representa.

- um nome ruim (`Interface1`, `Coisa`, `Processador2`) não diz nada sobre o que a classe promete fazer;
- um bom nome comunica a intenção do contrato antes mesmo de olharmos os métodos.

Duas convenções comuns para nomear interfaces

- **Adjetivos de capacidade**, terminados em `-vel` (equivalente ao `-able` do inglês): descrevem algo que a classe promete **saber fazer**;
- **Substantivos de papel**: descrevem **o que a coisa é** dentro do sistema, quando a interface representa um papel ou uma peça reutilizável.

Exemplos

`Pagavel` e `Comparable` seguem a primeira convenção. `List` e `Comparator` seguem a segunda.

Convenção 1: adjetivos de capacidade (-vel / -able)

- Runnable – a classe sabe **ser executada** por uma thread;
- Comparable – a classe sabe **se comparar** a outra do mesmo tipo;
- Serializable – a classe sabe **ser convertida** em bytes;
- Iterable – a classe sabe **ser percorrida** em um for-each;
- Cloneable – a classe sabe **ser clonada**.

Padrão

Todos são adjetivos: descrevem uma **capacidade** que a classe ganha ao assinar o contrato.

O nome já entrega o contrato

```
public interface Runnable {  
    void run();  
}
```

Observação

Só de ler o nome `Runnable`, já entendemos que implementar essa interface significa "ser algo que pode ser executado" – antes mesmo de olhar o método `run()`.

Por que "Pagavel"?

Aplicando a convenção

Pagavel segue exatamente o mesmo padrão de Runnable e Comparable: um adjetivo terminado em -vel.

- toda classe que implementa Pagavel promete ter a **capacidade de processar um pagamento**;
- o nome já entrega o contrato: "isso aqui é algo que pode ser pago" – sem precisar ler o método pagar para entender a intenção;
- repare que **nenhuma classe concreta aparece no nome** – Pagavel não menciona boleto, cartão ou Pix, porque a interface representa a capacidade, não uma implementação específica.

Convenção 2: substantivos de papel

- usada quando a interface representa **uma coisa** ou **um papel** dentro do sistema, não apenas uma ação isolada;
- exemplos: `List` (uma coleção ordenada), `Comparator` (uma estratégia de comparação), `EstrategiaXp` (uma estratégia de cálculo de XP).

Quando usar

Essa convenção é comum em interfaces usadas no padrão **Strategy**, onde o nome costuma vir com o sufixo/prefixo `Estrategia`.

Como escolher entre as duas convenções

Pergunta 1

"O que esse objeto promete **saber fazer?**" – se a resposta é um único verbo, tende ao adjetivo -vel (Pagavel, Comparavel, Executavel).

Pergunta 2

"Que **papel** esse objeto desempenha no sistema?" – se a resposta é "uma coleção", "uma estratégia", "um repositório", tende ao substantivo (List, EstrategiaXp, Repositorio).

Erros comuns de nomenclatura

- prefixo I (IPagavel) – convenção de outras linguagens, não é usada em Java;
- sufixo redundante (PagavelInterface) – o próprio tipo da declaração já deixa claro que é uma interface;
- nomes genéricos demais (Processavel, Gerenciavel) – não dizem nada sobre o domínio e podem significar qualquer coisa.

Lembre-se

Se o nome da interface não deixar claro o contrato sem olhar o código, provavelmente ele precisa melhorar.

Exercício rápido

Como você nomearia a interface de classes que sabem...

- ...voar? → `Voavel`
- ...ser impressas? → `Imprimivel`
- ...ser ordenadas por algum critério? → `Ordenavel` ou `Comparavel`
- ...notificar outros objetos? → `Notificavel`

Pensar no nome **antes** de escrever o método ajuda a entender qual é, de fato, o contrato.

Abstração vs. implementação, na prática

Olhe para esta linha de código bem conhecida:



Programa para a abstração, não para a implementação

Pense no **tipo declarado** (a abstração) como a promessa; a classe usada com `new` (a implementação) é só um jeito, entre vários, de cumprir essa promessa.

Regra prática para programar

Abstração

List, Pagavel

dizem **o que** um objeto sabe fazer.

Implementação

ArrayList, Boletto

dizem **como** isso é feito por dentro.

Pergunta para fazer sempre

Ao declarar uma variável, pergunte-se: "eu preciso saber **como** isso funciona por dentro, ou só **o que** isso promete fazer?" Na dúvida, declare pelo tipo da **abstração**.

E se eu precisar de algo que só o ArrayList tem?

Ideia natural: ArrayList tem métodos que List não tem, por exemplo:

```
List<String> lista = new ArrayList<>();  
lista.ensureCapacity(100); // erro de compilacao!
```

Por que isso "quebra"?

ensureCapacity não faz parte do contrato de List – só existe em ArrayList. Declarando a variável como List, você só enxerga o que o contrato promete.

Isso é má prática? Não!

É uma escolha, não um erro

Você decide o tipo da variável de acordo com o que realmente precisa usar.

- se você **só precisa** do que `List` promete → declare `List` (mais flexível, mais seguro);
- se você **realmente precisa** de algo específico do `ArrayList` → declare `ArrayList` direto, sem culpa.

Regra de ouro

Programa para a abstração **por padrão**. Só desça para a implementação concreta quando o seu código realmente depender de algo que só ela oferece.

Como decidir: List ou ArrayList?

Regra prática

Pergunte quais métodos você vai chamar naquela variável, durante o tempo em que ela existe no seu código.

- se **todos** os métodos que você usa estão em List → declare List;
- se **pelo menos um** método que você precisa só existe em ArrayList (como `ensureCapacity`) → declare ArrayList direto.

Isso não contradiz "programe para abstração"

É justamente aplicar o princípio direito: use a abstração **sempre que ela for suficiente**; dependa da implementação só quando você **realmente** depende dela.

Uma armadilha pior: fazer cast

Existe uma solução pior do que declarar `ArrayList` direto: fazer um **cast** escondido no meio do código.

```
List<String> lista = new ArrayList<>();  
((ArrayList<String>) lista).ensureCapacity(100);
```

Por que isso é pior

Isso esconde uma dependência real atrás de uma abstração falsa: o código **finge** que só precisa de `List`, mas na prática precisa de `ArrayList`.

Seja honesto na declaração

Se você precisa de algo específico do `ArrayList`, declare a variável como `ArrayList` desde o início – não esconda essa dependência atrás de um cast.

Recapitulando

- Uma **interface** é um **contrato**: define quais métodos uma classe promete ter, sem dizer como implementá-los;
- classes **sem relação de herança** podem compartilhar um mesmo contrato via `implements`;
- uma classe pode implementar **várias** interfaces ao mesmo tempo;
- **programar para a abstração** significa depender da interface (`List`, `Pagavel`), não da classe concreta (`ArrayList`, `Boleto`);
- o **nome** de uma interface deve comunicar a capacidade (`-vel/-able`, como `Pagavel`) ou o papel (`List`, `Comparator`) que ela representa.